

Linguagem de Programação I - Etapa 2 - Leonardo Farias de Oliveira

```
$$$ src.controle.projeto
```

```
from util.gerais import imprimir_objetos, imprimir_objeto, imprimir_objetos_internos,
imprimir_objetos_associacao_filtros
from util.data import Data
from entidades.seguradora import inserir_seguradora, Seguradora, get_seguradoras
from entidades.peca import Peca
from entidades.sinistro import inserir_sinistro, Sinistro, get_sinistros
from entidades.orcamento import criar_orcamento, get_orcamentos, selecionar_orcamento
```

```
def cadastrar_seguradoras():
```

```
    inserir_seguradora(Seguradora(nome='Porto Seguro', cidade='Dourados',
cobertura_percentual=90))

    inserir_seguradora(Seguradora(nome='Bradesco Seguros', cidade='Campo Grande',
cobertura_percentual=80))

    inserir_seguradora(Seguradora(nome='SulAmérica', cidade='Ponta porã',
cobertura_percentual=70))

    inserir_seguradora(Seguradora(nome='Mapfre', cidade='Dourados', cobertura_percentual=85))

    inserir_seguradora(Seguradora(nome='Allianz', cidade='Campo Grande',
cobertura_percentual=75))
```

```
def cadastrar_sinistros():
```

```
    sinistro = Sinistro(numero='1', cliente='Leon', telefone='67 99888-1100')
    inserir_sinistro(sinistro)

    sinistro.inserir_peca(Peca(código=11, nome='bandeja', categoria='OEM', preco=245,
mecânico_próprio=False))

    sinistro.inserir_peca(Peca(código=12, nome='pivo', categoria='OEM', preco=250,
mecânico_próprio=False))

    sinistro = Sinistro(numero='6', cliente='Scott', telefone='67 98988-1101')
    inserir_sinistro(sinistro)

    sinistro.inserir_peca(Peca(código=21, nome='amortecedor', categoria='Original', preco=310,
mecânico_próprio=False))

    sinistro.inserir_peca(Peca(código=22, nome='kit batente', categoria='Original', preco=90,
mecânico_próprio=False))

    sinistro = Sinistro(numero='11', cliente='Kennedy', telefone='67 96898-1000')
    inserir_sinistro(sinistro)

    sinistro.inserir_peca(Peca(código=31, nome='rolamento', categoria='Original', preco=180,
mecânico_próprio=True))
```

```
sinistro.inserir_pecas(Peca(código=33, nome='cubo', categoria='Original', preco=180,
mecânico_próprio=True))
```

```
sinistro = Sinistro(numero='19', cliente='Chris', telefone='44 90898-0100')
```

```
inserir_sinistro(sinistro)
```

```
sinistro.inserir_pecas(Peca(código=41, nome='terminal', categoria='Original', preco=90,
mecânico_próprio=True))
```

```
sinistro.inserir_pecas(Peca(código=44, nome='barra axial', categoria='Original', preco=90,
mecânico_próprio=True))
```

```
sinistro = Sinistro(numero='98', cliente='Redfield', telefone='66 91681-1198')
```

```
inserir_sinistro(sinistro)
```

```
sinistro.inserir_pecas(Peca(código=51, nome='disco freio', categoria='Genuína', preco=150,
mecânico_próprio=True))
```

```
sinistro.inserir_pecas(Peca(código=55, nome='pastilha freio', categoria='Genuína', preco=150,
mecânico_próprio=True))
```

```
def cadastrar_orcamentos():
```

```
    criar_orcamento(numero_sinistro='1', nome_seguradora='Porto Seguro', data=Data(dia=12,
mês=6, ano=2026))
```

```
    criar_orcamento(numero_sinistro='6', nome_seguradora='Bradesco Seguros', data=Data(dia=10,
mês=10, ano=2026))
```

```
    criar_orcamento(numero_sinistro='11', nome_seguradora='SulAmérica', data=Data(dia=3,
mês=8, ano=2026))
```

```
    criar_orcamento(numero_sinistro='19', nome_seguradora='Mapfre', data=Data(dia=2, mês=5,
ano=2026))
```

```
    criar_orcamento(numero_sinistro='98', nome_seguradora='Allianz', data=Data(dia=28, mês=4,
ano=2026))
```

```
if __name__ == '__main__':
```

```
    cadastrar_seguradoras()
```

```
    imprimir_objetos(cabeçalho= "\n Seguradoras: Nome - Cidade - Cobertura percentual",
objetos=get_seguradoras().values())
```

```
    cadastrar_sinistros()
```

```
    imprimir_objetos(cabeçalho= "\n Sinistros: Número - Cliente - Telefone percentual",
                    objetos=get_sinistros().values())
```

```
    print("\n Sinistros: Número - Cliente - Telefone percentual")
```

```
    print(' - Pecas: Nome - Categoria - Preco - Mecânico Próprio')
```

```
for índice, sinistro in enumerate(get_sinistros().values()):
```

```
    imprimir_objeto(índice=índice, objeto_str=sinistro)
```

```
    imprimir_objetos_internos(sinistro.pecas.values())
```

```
cadaststrar_orcamentos()
```

```
cabecalho_orcamento = ('Orcamento: Numero do Sinistro - Nome da Seguradora - Data do  
orcamento')
```

```
cabecalho_orcamento_filtros = (cabecalho_orcamento + '\n -- Cobertura Percentual - Telefone no  
Sinistro - Precos das pecas')
```

```
imprimir_objetos('\n' + cabecalho_orcamento, get_orcamentos())
```

```
filtros, orcamentos_selecionados = selecionar_orcamento()
```

```
imprimir_objetos_associção_filtros(cabecalho_orcamento_filtros, orcamentos_selecionados,  
filtros)
```

```
filtros, orcamentos_selecionados = selecionar_orcamento(data_mínima_orcamento=Data(dia=1,  
mês=5, ano=2026))
```

```
imprimir_objetos_associção_filtros(cabecalho_orcamento_filtros, orcamentos_selecionados,  
filtros)
```

```
filtros, orcamentos_selecionados = selecionar_orcamento(Data(1, 5, 2026),  
valor_máximo_peca=255)
```

```
imprimir_objetos_associção_filtros(cabecalho_orcamento_filtros, orcamentos_selecionados,  
filtros)
```

```
filtros, orcamentos_selecionados = selecionar_orcamento(Data(1, 5, 2026), 255,  
cobertura_mínima_seguradora=85)
```

```
imprimir_objetos_associção_filtros(cabecalho_orcamento_filtros, orcamentos_selecionados,  
filtros)
```

```
filtros, orcamentos_selecionados = selecionar_orcamento(Data(1, 5, 2026), 255, 85,  
prefixo_telefone_cliente=67)
```

```
imprimir_objetos_associção_filtros(cabecalho_orcamento_filtros, orcamentos_selecionados,  
filtros)
```

```
$$$ src.entidades.seguradora
```

```
seguradoras = {}
```

```
def get_seguradoras(): return seguradoras
```

```
def inserir_seguradora(seguradora):
```

```
    nome_seguradora = seguradora.nome
```

```
    if nome_seguradora not in seguradoras.keys():
```

```
        seguradoras[nome_seguradora] = seguradora
```

```
        return True
```

```
    else:
```

```
        print('Seguradora ' + nome_seguradora + ' tem cadastro')
```

```
        return False
```

```
class Seguradora:
```

```
    def __init__(self, nome, cidade, cobertura_percentual):
```

```
        self.nome = nome
```

```
        self.cidade = cidade
```

```
        self.cobertura_percentual = cobertura_percentual
```

```
    def __str__(self):
```

```
        formato = '{ } {:<18} { } {:<14} { } {:<3} { }'
```

```
        seguradora_formatada = formato.format('|', self.nome, '|', self.cidade, '|',  
str(self.cobertura_percentual), '|')
```

```
        return seguradora_formatada
```

\$\$\$ src.entidades.peca

class Peca:

def __init__(self, código, nome, categoria, preco, mecânico_próprio):

self.código = código

self.nome = nome

self.categoria = categoria if categoria in ('Original', 'Genuína', 'OEM') else 'indefinida'

self.preco = preco

self.mecânico_próprio = mecânico_próprio

def __str__(self):

if self.mecânico_próprio: mecânico_próprio_str = 'Mecânico Próprio |'

else: mecânico_próprio_str = ''

formato = ' {} {:<15} {} {:<12} {} {:<6} {} {}'

peca_formatado = formato.format('|', self.nome, '|', self.categoria, '|', self.preco, '|',
mecânico_próprio_str)

return peca_formatado

```
$$$ src.entidades.orcamento
```

```
from entidades.seguradora import get_seguradoras
```

```
from entidades.sinistro import get_sinistros
```

```
orcamentos = []
```

```
def get_orcamentos(): return orcamentos
```

```
def inserir_orcamento(orcamento):
```

```
    if orcamento not in orcamentos: orcamentos.append(orcamento)
```

```
    else: print ('Orcamento de Pecas tem cadastro --- ' + str(orcamento))
```

```
def criar_orcamento(numero_sinistro, nome_seguradora, data):
```

```
    sinistro = get_sinistros().get(numero_sinistro)
```

```
    if sinistro is None:
```

```
        print('Sinistro ' + numero_sinistro + ' não cadastrado')
```

```
        return
```

```
    seguradora = get_seguradoras().get(nome_seguradora)
```

```
    if seguradora is None:
```

```
        print('Seguradora ' + nome_seguradora + ' não cadastrada')
```

```
        return
```

```
    orcamento = Orcamento(sinistro, seguradora, data)
```

```
    inserir_orcamento(orcamento)
```

```
def selecionar_orcamento(data_mínima_orcamento=None, valor_máximo_pecas=None,  
cobertura_mínima_seguradora=None, prefixo_telefone_cliente=None):
```

```
    filtros = '\nFiltros -- '
```

```
    if data_mínima_orcamento is not None: filtros += ' Data mínima do orcamento: ' +  
str(data_mínima_orcamento)
```

```
    if valor_máximo_pecas is not None: filtros += ' Maior valor da peca: ' + str(valor_máximo_pecas)
```

```
    if cobertura_mínima_seguradora is not None: filtros += '\n - Cobertura mínima da seguradora: ' +  
str(cobertura_mínima_seguradora)
```

```
    if prefixo_telefone_cliente is not None: filtros += (' - DDD telefone cliente: ' +  
str(prefixo_telefone_cliente))
```

```

orcamentos_selecionados = []
for orcamento in orcamentos:
    if data_mínima_orcamento is not None and orcamento.data < data_mínima_orcamento:
        continue

    excluir_orcamento = False
    for peca in orcamento.sinistro.pecas.values():
        if valor_máximo_peca is not None and peca.preco > valor_máximo_peca:
            excluir_orcamento = True
            break
    if excluir_orcamento:
        continue

    if cobertura_mínima_seguradora is not None and orcamento.seguradora.cobertura_percentual
    < cobertura_mínima_seguradora:
        continue

    if prefixo_telefone_cliente is not None and not
    orcamento.sinistro.telefone.startswith(str(prefixo_telefone_cliente)):
        continue

    orcamentos_selecionados.append(orcamento)
return filtros, orcamentos_selecionados

```

```

class Orcamento:

```

```

    def __init__(self, sinistro, seguradora, data):
        self.sinistro = sinistro
        self.seguradora = seguradora
        self.data = data

    def __str__(self):
        formato = '{} {:<15} {} {:<19} {} {:<10} {}'
        mostra_formato = formato.format('|', self.sinistro.numero, '|', self.seguradora.nome, '|',
        str(self.data), '|')
        return mostra_formato

    def str_atributos_pecas(self):

```

```

atributos_pecas_str = "
for indice, peca in enumerate(self.sinistro.pecas.values()):
    if indice > 0:
        atributos_pecas_str += ' - '
        atributos_pecas_str += f'R$ {peca.preco:3d}'
return atributos_pecas_str

def str_filtro(self):
    formato = '{:>2} {} {:<11} {} {:<15} {}'
    filtro_formatado = formato.format(
        str(self.seguradora.cobertura_percentual),
        '|', self.sinistro.telefone, '|', self.str_atributos_pecas(), '|'
    )
    return self.__str__() + filtro_formatado

```

```
$$$ src.entidades.sinistro
```

```
sinistros = {}
```

```
def get_sinistros (): return sinistros
```

```
def inserir_sinistro(sinistro):
```

```
    numero_sisnitro = sinistro.numero
```

```
    if numero_sisnitro not in sinistros.keys():
```

```
        sinistros[numero_sisnitro] = sinistro
```

```
        return True
```

```
    else:
```

```
        print('Sinistro ' + numero_sisnitro + ' já tem cadastro')
```

```
        return False
```

```
class Sinistro:
```

```
    def __init__(self, numero, cliente, telefone):
```

```
        self.numero = numero
```

```
        self.cliente = cliente
```

```
        self.telefone = telefone
```

```
        self.pecas = {}
```

```
    def __str__(self):
```

```
        formato = '{} {:<3} {} {:<15} {} {:<14}'
```

```
        sinistro_formatado = formato.format('|', self.numero, '|', self.cliente, '|', self.telefone)
```

```
        return sinistro_formatado
```

```
    def inserir_peca(self, peca):
```

```
        nome_peca = peca.nome
```

```
        if nome_peca not in self.pecas.keys(): self.pecas[nome_peca] = peca
```

```
        else: print('Nome ' + nome_peca + ' já foi cadastrada em Sinistro')
```

```
$$$ src.util.data
```

```
from datetime import date
```

```
class Data:
```

```
    def __init__(self, dia, mês, ano):
```

```
        self.dia = dia
```

```
        self.mês = mês
```

```
        self.ano = ano
```

```
    def __str__(self):
```

```
        if self.dia < 10: data_str = '0' + str(self.dia)
```

```
        else: data_str = str(self.dia)
```

```
        if self.mês < 10: data_str += "/0" + str(self.mês) + "/"
```

```
        else: data_str += "/" + str(self.mês) + "/"
```

```
        data_str += str(self.ano)
```

```
        return data_str
```

```
    def __eq__(self, data):
```

```
        if self.dia == data.dia and self.mês == data.mês and self.ano == data.ano: return True
```

```
        return False
```

```
    def __ne__(self, data): return not self == data
```

```
    def __gt__(self, data):
```

```
        if self.ano > data.ano: return True
```

```
        elif self.ano < data.ano: return False
```

```
        if self.mês > data.mês: return True
```

```
        elif self.mês < data.mês: return False
```

```
        if self.dia > data.dia: return True
```

```
        elif self.dia < data.dia: return False
```

```
        return False
```

```
    def __lt__(self, data):
```

```
        if self.ano < data.ano: return True
```

```
        elif self.ano > data.ano: return False
```

```
        if self.mês < data.mês: return True
```

```
        elif self.mês > data.mês: return False
```

```
if self.dia < data.dia: return True
elif self.ano > data.ano: return False
return False
```

```
def __ge__(self, data):
    if self < data: return False
    else: return True
```

```
def __le__(self, data):
    if self > data: return False
    else: return True
```

```
def calcular_idade(self, data_referência=None):
    if data_referência is None:
        dia_atual_str, mês_atual_str, ano_atual_str = date.today().strftime("%d/%m/%Y").split('/')
        dia_referência, mês_referência, ano_referência = int(dia_atual_str), int(mês_atual_str),
int(ano_atual_str)
    else:
        dia_referência, mês_referência, ano_referência = data_referência.dia, data_referência.mês,
data_referência.ano
        idade = ano_referência - self.ano
        if mês_referência < self.mês or (mês_referência == self.mês and dia_referência < self.dia):
idade -= 1
    return idade
```

\$\$\$ src.util.gerais

```
def imprimir_objetos(cabeçalho, objetos, filtros=None):
```

```
    if filtros is not None: print(filtros)
```

```
    print(cabeçalho)
```

```
    for indice, objeto in enumerate(objetos):
```

```
        imprimir_objeto(indice, str(objeto))
```

```
def imprimir_objeto(indice, objeto_str):
```

```
    formato = '{ } { } { }'
```

```
    ordem = indice + 1
```

```
    separador = ' _ '
```

```
    string_formatado = formato.format(f'{ordem:2d}', separador, objeto_str)
```

```
    print(string_formatado)
```

```
def imprimir_objetos_associação_filtros(cabeçalho, objetos, filtros=None):
```

```
    if filtros is not None: print(filtros)
```

```
    print(cabeçalho)
```

```
    for índice, objeto in enumerate(objetos):
```

```
        imprimir_objeto(índice, objeto.str_filtro())
```

```
def imprimir_objetos_internos (objetos):
```

```
    for objeto in objetos: print(' -' + str(objeto))
```



Documento assinado digitalmente

LEONARDO FARIAS DE OLIVEIRA

Data: 28/04/2026 21:06:26-0300

Verifique em <https://validar.iti.gov.br>